License Plate Detection and Recognition on CCPD2019

Technical summary

10 November 2026

1 Problem and question

This project develops a pipeline that detects and recognizes Chinese license plates in CCPD2019 images. We evaluate two controlled improvements over an off-the-shelf baseline — fine-tuning a pretrained YOLO11s detector on our 5,000-image subset, and replacing the general-purpose EasyOCR engine with a task-specific convolutional recurrent neural network (CRNN) — and ask whether they improve end-to-end plate recognition.

2 Dataset

We built a reproducible subset of CCPD2019: 5,000 source images stratified by plate height and the two tilt values encoded in the filenames. Each source image was generated at three scales (1.0, 0.75, and 0.5), producing 15,000 image variants. The subset was split into 3,500 training, 700 validation, and 800 test images using seed 42, before resizing, so that all variants of one image remain in the same split and cross-scale data leakage is prevented.

Plate size categories used training-set height thresholds: small (≤ 83 px), medium (83–112 px), large (> 112 px). We converted the CCPD filename annotations into normalized YOLO boxes and plate crops with text labels. Three OCR preprocessing modes support the ablation: none, grayscale + CLAHE, and Canny. Primary experiments use original-scale images; downscaled variants support the scale-robustness evaluation.

3 Method

Detection. We used YOLO11s to locate license plates before recognition. The detector predicts a single class (license plate); the highest-confidence box is cropped and passed to the recognizer. As a baseline, we used the frozen pretrained detector from MorseTechLab (the Hugging Face repository, `license-plate-finetune-v1s.pt`). We fine-tuned it on 3,500 original-scale training images (640 px, batch 16, AdamW, learning rate 0.002) and selected the best checkpoint by validation mAP@0.5 (epoch 4 of 24).

Recognition. As a baseline, we used the frozen general-purpose EasyOCR engine. As a second condition, we trained a plate-specific CRNN (CNN + bidirectional LSTM + CTC, 2,999,419 parameters) from scratch: each crop is converted to grayscale and resized onto a 48×256 canvas while preserving its aspect ratio, and the classifier predicts the 58 training-set characters plus the CTC blank. Training used 3,500 original-scale crops without augmentation (50 epochs, Adam, learning rate 0.001, batch 32, seed 42); the epoch-39 checkpoint was selected by validation character accuracy. Preprocessing mode and confidence threshold (0.50) were selected on validation: no preprocessing.

4 Results

Table 1 summarizes the main test metrics (800 images and 800 crops).

Configuration	mAP@0.5	Detection F1	Character accuracy	Exact plates
Frozen YOLO11s + EasyOCR	0.9073	0.9167	0.3240	0 / 800
CRNN, ground-truth crops	—	—	0.7593	267 / 800
CRNN + frozen YOLO11s	0.9073	0.9167	0.6710	148 / 800
CRNN + fine-tuned YOLO11s	0.9949	0.9714	0.7661	296 / 800

Table 1: Test metrics of the evaluated configurations ($n = 800$).

Fine-tuning raised detection recall from 0.8938 to 0.9975 and precision from 0.9408 to 0.9466. On the ground-truth crops, CRNN outperformed EasyOCR with 0.7593 versus 0.3474 character accuracy and 267 versus 0 exact plates.

Preprocessing ablation. Table 2 reports the validation character accuracy of the three preprocessing modes; the selected policy used no preprocessing and a confidence threshold of 0.50 in all protocol groups.

Preprocessing mode	Character accuracy
No preprocessing	0.7558
Grayscale + CLAHE contrast	0.7469
Canny edge detection	0.4575

Table 2: Preprocessing ablation (validation character accuracy, CRNN).

Scale robustness. The 0.75 and 0.5 scale variants of the test set were evaluated with the same weights and the validation-selected policy, without retraining (Table 3).

Metric	1.0	0.75	0.5
Detection mAP@0.5 / F1, frozen	0.9073 / 0.9167	0.9218 / 0.9271	0.9294 / 0.9298
Detection mAP@0.5 / F1, fine-tuned	0.9949 / 0.9714	0.9949 / 0.9726	0.9949 / 0.9708
CRNN, ground-truth crops	0.7593 (267)	0.7613 (265)	0.7610 (268)
CRNN + frozen YOLO11s	0.6710 (148)	0.6713 (157)	0.6721 (144)
CRNN + fine-tuned YOLO11s	0.7661 (296)	0.7629 (297)	0.7555 (279)

Table 3: Test metrics by scale (recognition: character accuracy; exact plates in parentheses).

Both stages remain flat across scales: the fine-tuned detector keeps mAP@0.5 of 0.9949 at all three scales, and CRNN accuracy on the ground-truth crops varies within 0.7593–0.7613 because the recognizer resizes every crop to the fixed 48×256 canvas.

5 Interpretation

The end-to-end gap of the frozen detector to the OCR-only performance (0.6710 versus 0.7593 character accuracy) closes almost entirely after fine-tuning (0.7661): missed detections are the dominant source of end-to-end error. The task-specific CRNN beats the general-purpose EasyOCR, which recognized no plate exactly, and because every crop is resized to the fixed 48×256 canvas, recognition accuracy is set by crop content, not absolute image scale.

6 Failure analysis

We inspected representative failures from the test set and the preprocessing ablation (Table 4).

Stage (sample)	Observation	Likely cause
Frozen detector (`6f3c772c…`)	Large plate, 19° tilt, no prediction at any score	Weak prior for large plate aspect ratios in the pretrained model, especially at 10–19° tilt
Fine-tuned detector (`356b4ca6…`)	11 boxes: one true (0.84) and ten false positives	Fine-tuning removed the misses but left low-confidence false positives on cluttered scenes
CRNN recognition (`588e616d…`)	Ground truth 苏 `E050R8`, prediction `05oRo` 菲 (large, 19° tilt)	Fixed 48 × 256 input and training dominance of small, near-0° plates leave large tilted glyphs out of distribution
CRNN + Canny (`ce92ecf7…`)	Ground truth 浙 `JD389L`, prediction 皖 `AU0367` (small, 3° tilt)	The Canny edge map leaves only hollow stroke outlines; the lost fill and shading make the glyphs unreadable

Table 4: Representative failure cases and their likely causes.

The failures concentrate on large plates and the 10–19° tilt bin, where the OCR-only character accuracy drops to 0.5658 and 0.5193. Larger inputs and size/tilt-balanced training are the natural next steps.

7 Reproducibility

Code: the project repository.

Demo video: demo video (Google Drive).

Weights: the weights repository (fetched with SHA-256 verification). Python 3.12, single-threaded CPU inference; training 4,154.9 s (CRNN) and 1,439.8 s (detector); inference about 83–86 ms per image. Reproduction commands: `docs/md/final_results.md`.

8 Members' contributions

- **Andrei Polevoi** — Data & preprocessing: subset, size buckets, splits, labels, preprocessing module.
- **Maximilian Mifsud Bonici** — Detection: YOLOv11 baseline + fine-tuning, mAP evaluation, failure cases.
- **Evgenii Khovalyg** — Recognition: CRNN training + OCR baseline, character/plate metrics, fine-tuning.
- **Mikhail Krylov** — Pipeline & evaluation: end-to-end integration, thresholds, latency, sensitivity tables.